

Software Requirements Specification (SRS)

Project: LoneWolf – Live People Connection

Prepared by: Niv Badichi, Niv Horvitz

Date: 05/01/2026

Table of Contents

1. Introduction

- 1.1. Purpose
- 1.2. Document Conventions
- 1.3. Intended Audience and Reading Suggestions
- 1.4. Product Scope
- 1.5. References
- 1.6. Business Goals
- 1.7. Success Metrics

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. External Interface Requirements

- 3.1 User Interfaces
- 3.2 Hardware Interfaces
- 3.3 Software Interfaces
- 3.4 Communications Interfaces

4. System Features (Functional Requirements)

- 4.1 User Registration and Profile Management
- 4.2 Event Discovery and Interactive Map
- 4.3 Event Creation and Management
- 4.4 Real-Time Notifications and Alerts
- 4.5 Event Joining and Scheduling
- 4.6 Event Moderation and Reporting
- 4.7 Administrator Functions

4.8 Event Lifecycle Management

4.9 User Trust and Reputation

5. Other Nonfunctional Requirements

5.1 Performance Requirements

5.2 Usability Requirements

5.3 Reliability and Availability

5.4 Security Requirements

5.5 Privacy Requirements

5.6 Maintainability

5.7 Portability

5.8 Scalability Requirements

5.9 Legal and Regulatory Requirements

6. Logical Database Requirements

6.1 Data Entities

6.2 Relationships

6.3 Data Retention

6.4 Data Security

7. System Models

7.1 System Context Diagram (SCD) Description

7.2 Use Case Overview

7.3 Data Flow Description

7.4 Detailed Use Case Descriptions

7.4.1 Discover Events

7.4.2 Create Event

7.4.3 Join Event

8. Verification and Validation

8.1 Requirement Traceability

9. Appendices

Appendix A: Glossary

Appendix B: Acronyms and Abbreviations

Appendix C: Analysis Models (SCD)

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to formally define the functional and non-functional requirements for the LoneWolf system. This document serves as a binding agreement between stakeholders and the development team and provides a clear, unambiguous description of system behavior.

1.2 Document Conventions

- Shall indicate a mandatory requirement.
- Should indicate a recommended requirement.
- May indicate an optional feature.
- Requirements are uniquely numbered (REQ-x.y.z).
- Priority levels: High (P1), Medium (P2), Low (P3).

1.3 Intended Audience and Reading Suggestions

- Project Managers: Scope control and scheduling.
- Developers: System implementation guidance.
- QA Engineers: Test case derivation.
- Stakeholders: Validation of business needs.

1.4 Product Scope

LoneWolf is a mobile-based, city-oriented social activity platform that allows users to discover, create, and join real-world events. Unlike traditional social networks, LoneWolf emphasizes activities rather than profiles, reducing social pressure and encouraging organic interaction.

1.5 References

- IEEE 29148 – Systems and Software Requirements Specification
- Course Lecture 6 – Writing the SRS
- SmartHome SRS Example
- LoneWolf Project One-Pager

1.6 Business goals

- Fast event discovery and decision making
- Increase meaningful participation, filter only to specific user-related events
- Improve user safety and trust, through authentication, reporting and other meaningful trust indicators
- Empower hosts to create and manage quality events
- Strengthen community growth through positive reviews and feedbacks from events, maintaining a consistent flow of events
- Maintain reliable service with predictable performance, supporting growth without degrading user experience.

1.7 Success metrics

- Time-to-First-Relevant-Event (TTFRE): Median time from first app open to viewing an event that matches the user's city/radius and chosen category filters. *Target: ≤ 60 seconds.*
 - Search-to-View Conversion: % of sessions with filters/map usage that result in at least one event details view. *Target: $\geq 70\%$.*
 - Event Listing Coverage (City Availability): % of active users who have at least N events available within a configured radius/time window. *Target: $\geq 80\%$ have ≥ 5 events available.*
 - View-to-Join Conversion: % of users who join an event after viewing its details page. *Target: $\geq 12\%$ (initial), trend upward over time.*
 - Event Completion Rate: % of events that reach "Completed" status (not canceled and not removed). *Target: $\geq 85\%$.*
 - Event fill rate: Average % of capacity filled for events that start (joins/capacity). *Target: $\geq 40\%$ (initial), category-dependent.*
 - Moderation Response Time: Median time from report submission to first moderator/admin action. *Target: ≤ 24 hours (median).*
 - *Join success rate:* % of join attempts that complete successfully without error. *Target: $\geq 99\%$.*
 - Daily active users and monthly active users: Unique users that are active in a week/month. *Target: trend post-launch.*
-

2. Overall Description

2.1 Product Perspective

LoneWolf is a standalone system consisting of:

- Mobile Client Application (iOS & Android)
- Backend Server
- Cloud Database
- External Map and Notification Services

2.2 Product Functions

Major system functions include:

1. User registration and profile management
2. Location-based event discovery
3. Event creation and hosting
4. Real-time notifications
5. Event participation and scheduling
6. Reporting and moderation

2.3 User Classes and Characteristics

- Regular User: Searches and joins events.
- Event Host: Creates and manages events.
- Administrator: Oversees system integrity and moderation.

2.4 Operating Environment

- Mobile OS: Android, iOS
- Backend: Cloud-based (AWS/Firebase)

- Network: Internet connectivity with GPS enabled

2.5 Design and Implementation Constraints

- GDPR compliance for location data
- Dependence on third-party map APIs
- Battery efficiency constraints

2.6 User Documentation

- In-app onboarding tutorial
- Help & FAQ section

2.7 Assumptions and Dependencies

- Users own GPS-enabled smartphones
 - External services remain available
-

3. External Interface Requirements

3.1 User Interfaces

- Interactive Map View
- Event Feed View
- Event Creation Form
- Profile Management Screen

3.2 Hardware Interfaces

- GPS sensor
- Touchscreen input

3.3 Software Interfaces

- Google Maps / Apple MapKit
- Push Notification Services
- Calendar APIs

3.4 Communications Interfaces

- HTTPS for all data transfers
 - Encrypted push notification channels
-

4. System Features (Functional Requirements)

4.1 User Registration and Profile Management

- REQ-4.1.1: The system shall allow user registration via email or social login.
- REQ-4.1.2: The system shall store user interests.
- REQ-4.1.3: The system shall allow profile editing.

4.2 Event Discovery and Interactive Map

- REQ-4.2.1: The system shall display nearby events on a map.
- REQ-4.2.2: The system shall support category filters.

4.3 Event Creation and Management

- REQ-4.3.1: Users shall create events with title, time, location, and capacity.
- REQ-4.3.2: Hosts shall update or cancel events.

4.4 Real-Time Notifications and Alerts

- REQ-4.4.1: Users shall receive reminders before events.
- REQ-4.4.2: Users shall receive arrival-time alerts.

4.5 Event Joining and Scheduling

- REQ-4.5.1: Users shall join events if capacity allows.
- REQ-4.5.2: Events may be exported to calendars.

4.6 Event Moderation and Reporting

- REQ-4.6.1: Users shall report inappropriate events.
- REQ-4.6.2: Admins shall review reports.

4.7 Administrator Functions

Administrators are responsible for maintaining system integrity, user safety, and content quality. Administrative functions are not exposed to regular users and require elevated permissions.

- REQ-4.7.1: The system shall provide administrators with a secure administrative dashboard.
- REQ-4.7.2: Administrators shall be able to view all reported events and user accounts.

- REQ-4.7.3: Administrators shall be able to remove events that violate platform guidelines.
 - REQ-4.7.4: Administrators shall be able to suspend or permanently ban users.
 - REQ-4.7.5: The system shall log all administrative actions for auditing purposes.
 - REQ-4.7.6: Admins shall remove inappropriate content.
 - REQ-4.7.7: Admins shall manage user bans.
-

4.8 Event Lifecycle Management

This section describes the full lifecycle of an event from creation to completion.

- REQ-4.8.1: The system shall mark events as *Active*, *Ongoing*, *Completed*, or *Canceled*.
 - REQ-4.8.2: The system shall automatically transition an event to *Ongoing* when its start time is reached.
 - REQ-4.8.3: The system shall archive completed events after 24 hours.
 - REQ-4.8.4: Archived events shall remain accessible for analytics and user history.
-

4.9 User Trust and Reputation

To encourage positive interactions, the system maintains a lightweight reputation model.

- REQ-4.9.1: The system shall allow users to provide feedback after participating in an event.
 - REQ-4.9.2: Feedback shall be aggregated into a non-numeric trust indicator.
 - REQ-4.9.3: Reputation indicators shall not expose detailed personal ratings.
 - REQ-4.9.4: the system shall allow users attending events that are only corresponding to their current VPN location.
-

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- REQ-5.1.1: Map loading shall not exceed 2 seconds.
- REQ-5.1.2: System shall support 10,000 concurrent users per city.

5.2 Usability Requirements

- REQ-5.2.1: Core actions shall be achievable within three clicks.
- REQ-5.2.2: UI shall be accessible and minimalistic.

5.3 Reliability and Availability

- REQ-5.3.1: System uptime shall be 99.9%.

5.4 Security Requirements

- REQ-5.4.1: All user data shall be encrypted.

- REQ-5.4.2: Secure authentication shall be enforced.

5.5 Privacy Requirements

- REQ-5.5.1: Location data shall not be shared without consent.

5.6 Maintainability

- REQ-5.6.1: Modular architecture shall be used.

5.7 Portability

- REQ-5.7.1: The system shall support both Android and iOS platforms.
 - REQ-5.7.2: The backend shall be deployable on any cloud infrastructure compliant with container-based deployment.
 - REQ-5.7.3: The system should allow future expansion to web-based clients without architectural redesign.
-

5.8 Scalability Requirements

- REQ-5.8.1: The system shall scale horizontally to support growth in users per city.
- REQ-5.8.2: The system shall allow independent scaling of map services, notification services, and user management modules.

5.9 Legal and Regulatory Requirements

- REQ-5.9.1: The system shall comply with GDPR regulations regarding personal and location data.
- REQ-5.9.2: The system shall provide users with the ability to delete their accounts and associated data.
- REQ-5.9.3: The system shall store audit logs for administrative actions for a minimum of 12 months.
- The application shall support iOS and Android.

5.10 Localization & Internationalization

5.11.1: The system shall support multiple languages.

5.11.2: All textual content shall be externalized for translation.

5.11.3: Time and date formats shall adapt to user locale

6. Logical Database Requirements

The LoneWolf system relies on a centralized database to store operational and historical data. The database design shall support scalability, security, and efficient querying.

6.1 Data Entities

The primary data entities include:

- 6.1.1:User: Stores user profile information, preferences, and account status.

- 6.1.2:Event: Stores event metadata including title, time, location, capacity, and status.
- 6.1.3:Participation: Represents user participation in events.
- 6.1.4:Notification: Stores notification delivery records.
- 6.1.5:Report: Stores user-submitted reports and moderation outcomes.

6.2 Relationships

- 6.2.1:A User may create zero or more Events.
- 6.2.2:A User may participate in multiple Events, with the fact that the events are not overlapping each other.
- 6.2.3:An Event may have multiple Participants.
- 6.2.4:Reports are associated with Users and Events.

6.3 Data Retention

- 6.3.1:User account data shall persist until account deletion.
- 6.3.2:Event data shall persist for a minimum of 12 months.
- 6.3.3:Logs and audit records shall persist for a minimum of 24 months.

6.4 Data Security

- 6.4.1:All personally identifiable information shall be encrypted at rest.
 - 6.4.2:Access to administrative data shall be role-based.
 - 6.4.3:User Profiles
 - 6.4.4:Events
 - 6.4.5:Event Participation
 - 6.4.6:Notifications
 - 6.4.7:Activity Logs
-

7. System Models

7.1 System Context Diagram (SCD) Description

The System Context Diagram illustrates LoneWolf as a central system interacting with external entities including users, mapping services, calendar services, and notification providers. Users interact exclusively through mobile clients, while third-party services provide location, routing, and notification delivery.

7.2 Use Case Overview

Primary use cases include:

- User Registration
- Discover Events
- Create Event
- Join Event
- Abort event attandment

- Add event to calendar
- Receive Notification
- Report Event

Each use case follows a simple and direct interaction flow, consistent with the platform's low social pressure philosophy.

7.3 Data Flow Description

User actions generate requests that are validated by the backend server, processed according to business rules, and persisted in the database. Outbound notifications and map updates are triggered asynchronously.

7.4 Detailed Use Case Descriptions

7.4.1 Use Case: Discover Events

Actors: Registered User

Preconditions:

- User is authenticated
- GPS permissions granted

Main Flow:

- User opens the application
- System retrieves current location

- System displays nearby events on the interactive map
- User applies filters (category, time, distance)
- System updates event list in real time

Alternative Flow:

- If GPS is disabled, system prompts user to enable location services

Postconditions:

- Relevant events are displayed to the user
-

7.4.2 Use Case: Create Event

Actors: Event Host

Preconditions:

- User is authenticated

- User has hosting privileges

Main Flow:

- Host selects “Create Event”
- Host enters event details (title, location, time, capacity)
- System validates input
- Event is stored and marked as Active

Postconditions:

- Event becomes visible to nearby users
-

7.4.3 Use Case: Join Event

Actors: Registered User

Preconditions:

- Event is Active
- Capacity not exceeded

Main Flow:

- User selects an event
- User clicks “Join”
- System verifies capacity and schedule conflicts
- Participation is recorded

Postconditions:

- User is added to participant list

8. Verification and Validation

Each requirement defined in this document shall be subject to verification using one or more of the following methods:

- Inspection: Manual review of requirement implementation.
- Demonstration: Live execution of system functionality.

- Testing: Automated or manual test cases.
- Analysis: Logical reasoning or simulation.

8.1 Requirement Traceability

All functional and non-functional requirements shall be traceable to test cases to ensure full coverage.

Each requirement shall be validated through testing, inspection, or demonstration.

9. Appendices

Appendix A: Glossary

- Event: A scheduled real-world activity.
- Host: User who creates an event.
- User: An authenticated person, registering himself to the system.

- Admin: A pre anointed person that has several authorities for managing system updates, report cases. Can be a dev team, stakeholder, or other entities of interest.

Appendix B: Acronyms and Abbreviations

- SRS: Software Requirements Specification
- GPS: Global Positioning System
- GDPR: General Data Protection Regulation
- UI: User Interface
- API: Application Programming Interface
- AWS: Amazon Web Services
- HTTPS: Hypertext Transfer Protocol Secure
- SCD: System Context Diagram
- iOS: Apple Mobile Operating System

Appendix C: Analysis Models (SCD)

